Release Sooner, Expose Less: Offloading MoE Dispatch in NCCL-EP under Dual-Batch Overlap

Hsin-Chun Yin, Khaled Hamidouche, Ian Ziemba, Arjun Kashyap, Pak Markthub,
Subhadeep Bhattacharya, Ke Wen, Assaf Ravid, Manjunath Gorentla Venkata, and Gil Bloch
NVIDIA
{hsinchuny, khamidouche, iziemba, arjkashyap, pmarkthub,
subhadeepb, kwen, aravid, manjunath, gil}@nvidia.com

arXiv:2611.12362v1 [cs.DC] 9 Nov 2026

***Abstract*—Dual-batch overlap (DBO) hides mixture-of-experts (MoE) communication behind another microbatch's computation, but only after dispatch send releases the GPU. The time dispatch send holds the GPU therefore stays exposed, while network progress after release can be hidden. We evaluate three research configurations built on NCCL-EP that target this exposed communication time: GPI, in which the GPU constructs GPU-Friendly Descriptors (GFDs) and submits them to NIC-accessible memory for transport; GPI-MC, which describes MoE token fan-out in one GFD; and GDAKI Free Flow, which moves send progression to a host thread. On a GB300 NVL72 system, all three approaches reduce dispatch-send time relative to default GDAKI over most operating points in the evaluated parameter sweep. An experiment that inserts a GPU delay between dispatch send and receive shows that offload increases the in-flight interval. Depending on the configuration, 32–512 μs of computation or other independent work between send and receive hides the deferred progress, leaving less exposed time than default GDAKI. In disaggregated vLLM serving with DBO, these approaches lower median TPOT by 15–25% relative to NCCL-EP default GDAKI and by 16–42% relative to DeepEP v1.**

***Index Terms*—mixture of experts, GPU-initiated networking, communication overlap, dual-batch overlap, RDMA, GPI, GDAKI, network offload, vLLM**

I. Introduction

Sparse mixture-of-experts (MoE) models route each token to a small subset of experts distributed across accelerators [1]–[3]. Their dispatch and combine operations are irregular all-to-all exchanges whose many small, dynamic transfers make submission latency important during inference. Under dual-batch overlap (DBO), submission latency is particularly important: once dispatch send releases the GPU, another microbatch can compute while the first continues through the host, NIC, and fabric.

NCCL-EP's default GDAKI path uses GPU-initiated networking, which removes a CPU launch but can still block the sending SM during WQE construction, publication, doorbells, credit waits, and ordering. This paper seeks to release the GPU earlier and thereby reduce exposed communication under DBO, without requiring earlier network completion.

© 2026 NVIDIA. All rights reserved.

We study three research configurations built on NCCL-EP. With GPI, the GPU constructs a GPU-Friendly Descriptor (GFD), submits it to NIC-accessible memory, and returns while the NIC realizes the requested transport operation. GPI multicast (GPI-MC) refines this submission boundary for MoE dispatch, where a token often goes to several destinations, by letting the GPU describe the token's fan-out once rather than once per destination. GDAKI Free Flow keeps WQE construction on the GPU but moves send progression to a host thread. We evaluate these configurations with NCCL-EP dispatch-send microbenchmarks and with vLLM time per output token (TPOT) under DBO.

The central mechanism is that *earlier dispatch-send release can reduce exposed communication even with unchanged or slower network completion, provided useful work covers the deferred progress*. In disaggregated vLLM serving at loads where DBO splits reliably, the GPI, GPI-MC, and GDAKI Free Flow configurations show 15–25% lower median TPOT relative to NCCL-EP default GDAKI and 16–42% lower median TPOT relative to a DeepEP v1 default-GDAKI baseline. The dispatch-send microbenchmarks and end-to-end vLLM serving measurements come from separate experiments; their correlation is consistent with the proposed mechanism but does not by itself establish dispatch-send duration as the isolated cause of the TPOT changes.

II. Background and Network-Side Design

A. MoE Dispatch under Data and Expert Parallelism

A mixture-of-experts (MoE) layer routes each token to a small subset of its experts [1]. In the deployments considered here, both the data-parallel (DP) and expert-parallel (EP) dimensions exceed one: DP ranks own different token batches, while the experts are partitioned across EP ranks. A token's selected expert therefore need not be on the GPU that owns the token.

After routing, the distributed MoE layer has three logical stages. Dispatch moves token activations to the ranks that own their selected experts. Those experts execute their GEMMs. Combine then returns the weighted expert outputs to the source ranks and restores token order.

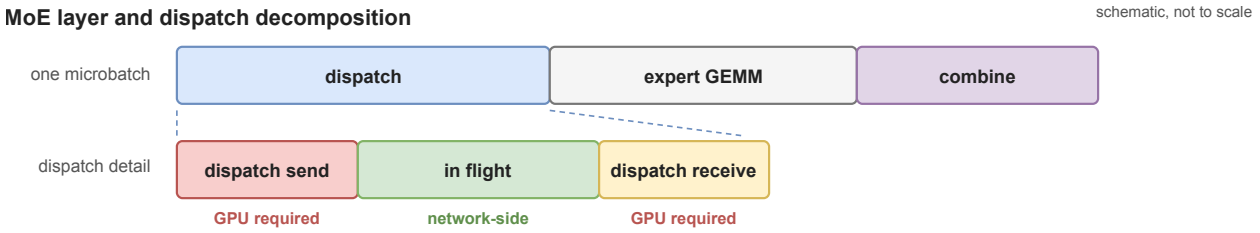


Fig. 1. An expert-parallel MoE layer. Each microbatch executes dispatch, expert GEMM, and combine. Dispatch consists of GPU-required send and receive intervals separated by network-side in-flight progress. Lengths are schematic.

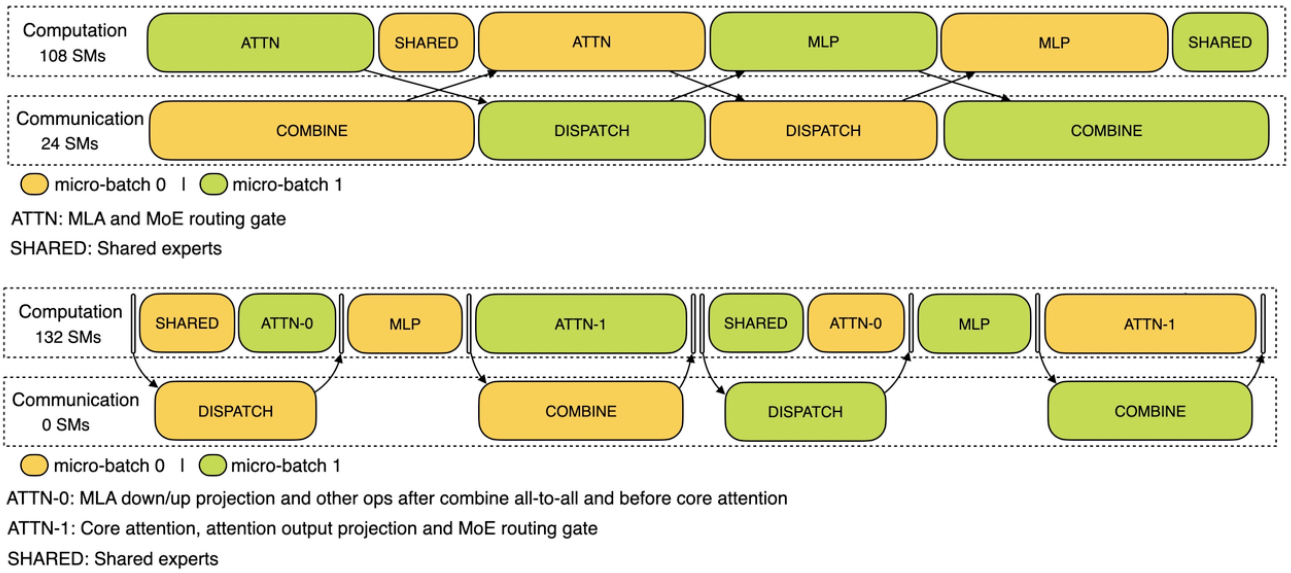


Fig. 2. DeepSeek's original DBO schedules for prefill (top) and decode (bottom). Orange and green identify the two microbatches. Prefill alternates their attention, shared-expert, MLP, dispatch, and combine stages. Decode splits attention into two parts to form a five-stage pipeline with dispatch and combine. In both schedules, communication for one microbatch runs under computation for the other. Reproduced from DeepSeek-AI [4] (CC0 1.0).

Figure 1 expands dispatch into three intervals. *Dispatch send* is the source-GPU work that prepares any required payloads, incorporates routing information when needed, constructs transfer requests, and submits the transfers. Payload preparation may include packing, but packing is not required by this definition. *In flight* is the remaining network-side progress after the source GPU is released. *Dispatch receive* is the destination-GPU work that waits for arrivals and makes the received tokens ready for expert computation. These are logical ownership boundaries: a transfer may begin before the send kernel returns.

B. Dual-Batch Overlap and Exposed Communication Time

Dual-batch overlap (DBO) splits a batch into two microbatches and schedules communication from one microbatch beneath computation from the other. DeepSeek describes this schedule for both prefill and decode as a way to hide cross-node EP communication [4].

Figure 2 shows the complete schedules that motivate our model. In prefill, the longer attention and MLP regions provide covering windows for dispatch and combine. Decode has less uniform stage durations, so DeepSeek splits each microbatch's attention layer into two steps: ATTN-0 (the projections before core attention) and ATTN-1 (core attention, output projection, and the MoE gate). Together with shared-expert and MLP computation, these steps form a five-stage pipeline in which each stage of one microbatch covers communication of the other.

Our serving experiments exercise the decode schedule. To identify what a network backend can improve within that schedule, we next expand one communication stage into its GPU-required send and receive intervals and its network-side in-flight interval.

In this paper, *GPU-side submission* denotes a path that keeps transfer submission and subsequent queue progression on the sending GPU's SMs. *Network-side submission* denotes a path in which the sending GPU first publishes the transfer request and, after that publication, the NIC or a host progress thread performs later progression, allowing the sending GPU to return earlier. Both terms describe control ownership after request preparation; they do not imply different payload or

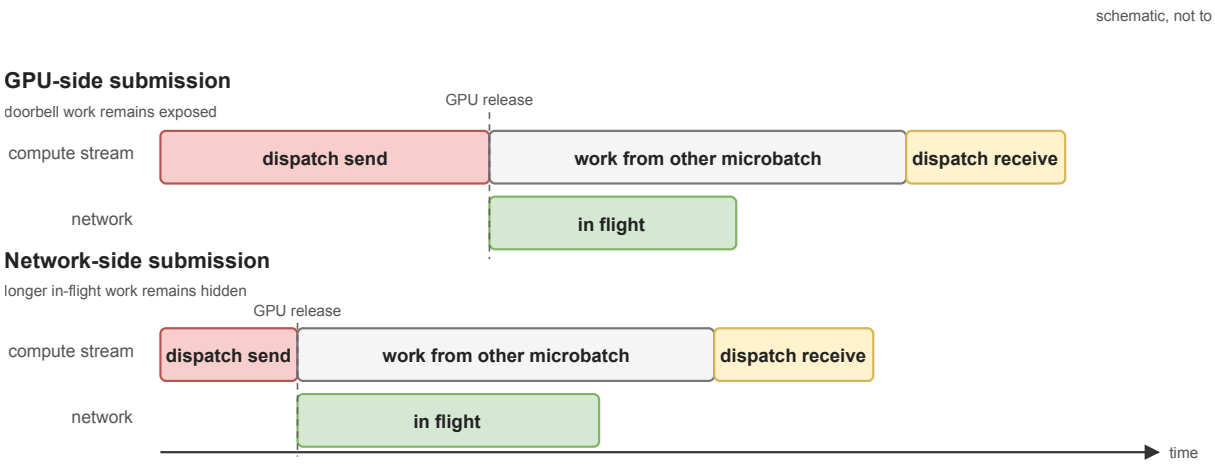


Fig. 3. Earlier dispatch-send return improves a two-microbatch DBO schedule. The white block is work from the other microbatch. Moving later submission progression away from the sending SM shortens the red dispatch-send interval and therefore the exposed communication time, starts the covering work earlier, and keeps the green in-flight interval hidden beneath it. The network-side path therefore reaches dispatch receive and completes the MoE layer earlier even though its illustrated in-flight interval is slightly longer. Both rows share one schematic time scale.

receiver semantics.

Figure 3 compares two schedules for the same two-microbatch MoE layer. In each row, the red send and yellow receive belong to one microbatch, while the white block is useful GPU work from the other microbatch. The green interval begins exactly when dispatch send releases the GPU. With GPU-side submission, the longer send delays both the covering work and the eventual receive. Network-side submission returns earlier, so the other microbatch starts sooner and hides the remaining network progress. The lower green interval is deliberately slightly longer than the upper one, yet it remains shorter than the unchanged white work interval: network completion need not improve for the MoE layer to finish earlier.

Exposed communication time is the portion of communication that remains on the application critical path after all available computation overlap. In Fig. 1, dispatch send and dispatch receive are exposed because they require the GPU needed by the covering computation. Once dispatch send releases that GPU, network-side progress can run beneath the other microbatch. In-flight progress contributes to exposed communication time only when it extends beyond the covering work.

Measure all timestamps from the start of dispatch send. Let A be the timestamp at which useful work from the other microbatch is ready, and let R_b be the timestamp at which backend b releases the sending GPU. The work can start only after both events, so its start timestamp is $S_b = \max(A, R_b)$; the maximum selects the later event. Thus, A, R_b, and S_b are points in time, not durations.

For the baseline and offloaded paths, assume $R_{\text{off}} \le R_{\text{base}}$. Define $\Delta_R \equiv R_{\text{base}} - R_{\text{off}}$ as the duration by which offload advances GPU release, and $\Delta_S \equiv S_{\text{base}} - S_{\text{off}}$ as the duration by which the covering work starts earlier. Then

$$0 \le \Delta_S \le \Delta_R. \tag{1}$$

The advance is full when $A \le R_{\text{off}}$, partial when $R_{\text{off}} < A < R_{\text{base}}$, and zero when $A \ge R_{\text{base}}$. Thus, earlier GPU release can advance the covering work by at most the release-time reduction; it need not advance it at all. Equation (1) therefore describes only when covering work can start, not how much network progress it hides.

To model that overlap, let I_b be backend b's in-flight duration from R_b until the final arrival required by dispatch receive, and let T_{cover} be the duration of the other microbatch's covering work. At S_b, the remaining in-flight duration J_b divides into an overlapped portion O_b and an exposed tail X_b:

$$\begin{aligned} J_b &= [R_b + I_b - S_b]_+, \\ O_b &= \min(T_{\text{cover}}, J_b), \qquad X_b = [J_b - T_{\text{cover}}]_+, \end{aligned} \tag{2}$$

where $[x]_+ = \max(x, 0)$. Dispatch receive can begin at

$$D_b = \max(S_b + T_{\text{cover}}, R_b + I_b) = S_b + T_{\text{cover}} + X_b. \tag{3}$$

When covering work starts at GPU release, $S_b = R_b$ and $J_b = I_b$. If $T_{\text{cover}} \ge I_b$, all in-flight progress overlaps and $X_b = 0$. If $T_{\text{cover}} < I_b$, the uncovered duration is $X_b = I_b - T_{\text{cover}}$. When $S_b > R_b$, some network progress occurs before the covering work starts, so T_{cover} need only cover J_b, not the full I_b. Dispatch send, dispatch receive, and any nonzero X_b remain exposed. Reducing dispatch send can therefore shorten the layer even when offload increases I_b, provided the covering work keeps X_b small enough. Whether DBO supplies enough covering work is an empirical question, which Sections IV-B and IV-C examine.

C. GIN and Its Backend Paths

GPU-Initiated Networking (GIN) is NCCL's GPU-facing network abstraction for operations such as device-side memory windows, puts, signals, and completions. NCCL-EP uses those

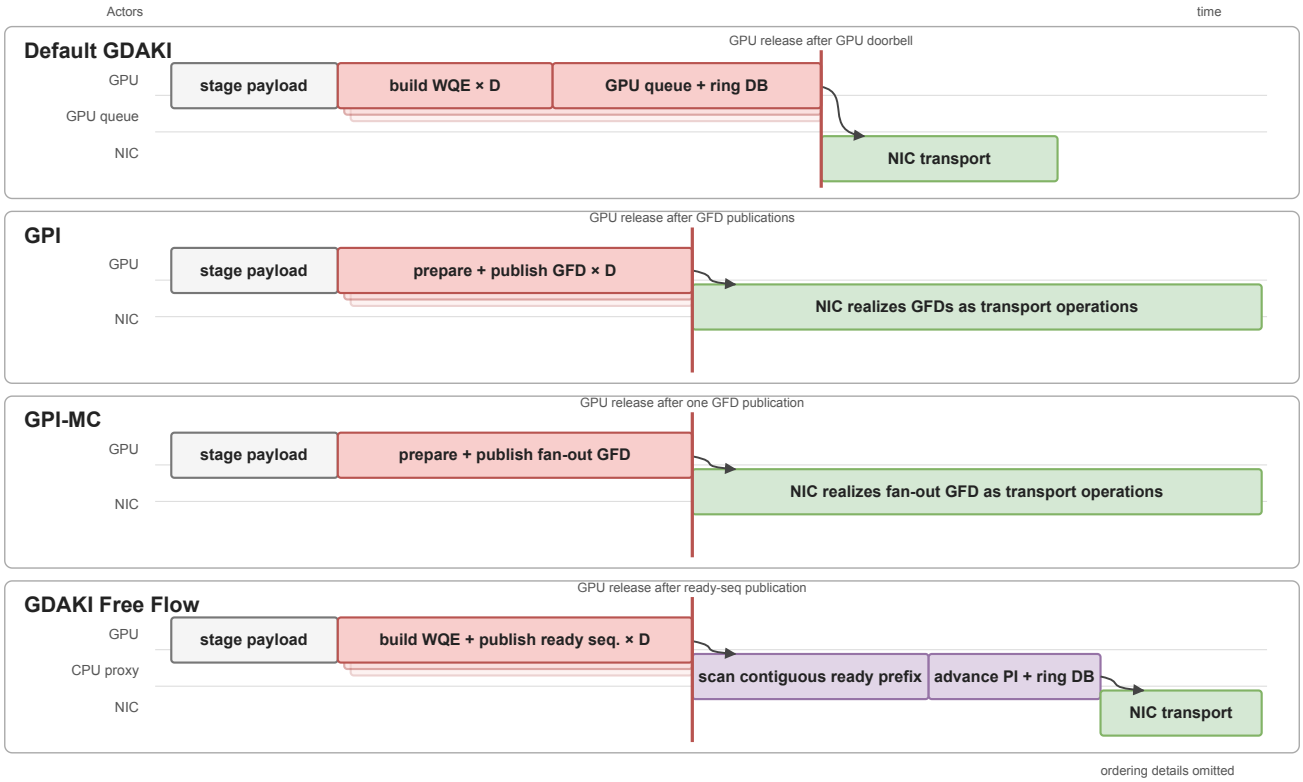


Fig. 4. The two GIN backends and their evaluated optimizations on a common actor/time layout. Red spans denote the dispatch-send component of exposed communication time and are schematic: default GDAKI is longer, while GPI, GPI-MC, and GDAKI Free Flow share the same earlier release boundary. Stacked red boxes in default GDAKI, GPI, and GDAKI Free Flow denote repeated per-destination WQE or GFD operations, parallelized across GPU threads. GPI-MC alone submits one fan-out GFD from the GPU. For both GPI paths, the single green block represents NIC transport after GFD submission. GDAKI Free Flow instead uses a host progress thread after the GPU publishes WQE readiness. Ordering fences required by every path are omitted.

operations to implement the remote transfers in dispatch and combine while selecting among transport backends [5], [6].

We study GIN's GDAKI and GPI backends and one optimization of each. Default GDAKI builds RDMA work-queue entries (WQEs) in GPU memory and performs the queue progression and NIC doorbell from the GPU. GDAKI Free Flow keeps the GPU-built WQE but moves ready-prefix progression and the doorbell to a host proxy. Both GDAKI paths repeat GPU-side WQE preparation and publication across destination transfers; the stacked cards in Fig. 4 show that these operations are parallelized, with many GPU threads collaboratively posting the sends. GPI instead has the GPU construct a GPU-Friendly Descriptor (GFD) and submit it to NIC-accessible descriptor memory. The NIC consumes the GFD and realizes the requested transport operation. GPI-MC is an experimental MoE-specific GPI optimization: it submits one special GFD describing a token's fan-out rather than one ordinary GFD per remote destination.

1) Default GDAKI and GDAKI Free Flow: Default GDAKI places the send queue and WQEs in GPU memory, and GPU code manages the queue and rings the NIC doorbell directly [5], [7]. In the tested NCCL version (2.31.2), a sending SM reserves queue space, writes the WQE, advances the send-queue producer state, serializes shared-QP state, and rings the NIC. The NIC then fetches the WQE and payload. Queue management and the doorbell therefore remain part of dispatch send.

GDAKI Free Flow changes control ownership without changing the payload path. The GPU still reserves a send-queue slot and writes the complete WQE, but then release-orders that slot's absolute sequence number into its ready-array entry and returns. Each ready-array entry marks one individual queue slot ready; the absolute sequence number prevents a stale marker from a previous queue lap from validating a reused slot. A host progress thread derives the *ready prefix*, defined as the longest contiguous sequence of ready entries beginning at the current producer frontier, then advances that frontier through the prefix and rings the NIC. Thus, the ready prefix is derived queue state, not an element of the ready array. This design shortens the GPU-resident send path by shifting queue progression to the host, and therefore relies on continuously available CPU progress capacity.

2) GPI and the GPI-MC refinement: For an ordinary GPI write, the GPU stages the payload, constructs one GFD for each distinct remote destination, and submits each GFD to NIC-accessible descriptor memory. The NIC then realizes each descriptor as a transport operation. At high token count or fan-out, issuing many per-destination descriptors can increase

TABLE I
SOFTWARE STACK AND RUNTIME SETTINGS.

Component	Version or setting
vLLM	0.30.1rc0-based fork, one-stream DBO
NCCL with GIN	2.31.2-based research forks
MoE integration	Research prototype
DeepEP baseline	v1.2.1-based LL research fork
Model	Qwen3-30B-A3B `ad44e777`, BF16
Platform	GB300-CX8 NVL72, aarch64, CUDA 13.0
GIN contexts/rank	4
Serving SM cap	64

GPU-visible submission time.

GPI-MC targets this MoE-specific per-destination submission overhead. The GPU publishes a single special GFD containing the token's fan-out plan, and the NIC realizes the requested per-destination transfers. By reducing the number of GFDs that the GPU must submit, GPI-MC aims to preserve GPI's low-token dispatch-send behavior at larger token counts.

All four paths enforce the two orderings that dispatch correctness requires: each payload is staged before submission, whether submission publishes a GFD or writes a WQE and rings its doorbell; each payload transfer then precedes the remote signal that announces it under the transport's ordering semantics. Violating either would let the receiver read incomplete data. The optimizations change only where send-side work progresses, not these orderings or the receiver-visible dispatch semantics.

III. METHOD

We use three sets of experiments. First, we measure how long each backend keeps the GPU busy during dispatch send. Second, we insert a controlled delay between dispatch send and receive to measure how much time is needed to hide the remaining network work. Third, we run vLLM with DBO and measure end-to-end TPOT.

A. NCCL-EP Experiments

Both experiments use NCCL-EP's low-latency path with BF16 activations, hidden dimension 2048, 128 experts, four GIN contexts per rank, and NVLink disabled. We evaluate default GDAKI, GPI, GPI-MC, and GDAKI Free Flow. Every cell uses equal token counts on all ranks and random top-k expert selection without replacement. The routing is computed once per cell and reused for all timed iterations and rounds, so the sweeps do not vary routing skew. Table I lists the software versions and runtime settings. The dispatch factor sweep keeps each backend and its reference on the same allocation within a recorded shuffled shape block. The wait sweep uses one common two-node allocation for all four backends in each round, with backend order shuffled reproducibly and node groups allowed to vary between rounds.

a) Experiment 1: dispatch-send factor sweep: This experiment asks how much each backend reduces dispatch-send time, the first component of exposed communication time, across MoE operating points. It crosses GPU count $\{4, 8, 16\}$, dispatch SM cap $\{32, 64, 128\}$, top-$k \in \{4, 8\}$, and tokens/rank $\{1, 2, 4, 8, 16, 32, 64, 128, 256\}$. Each iteration launches dispatch send, an 8192-square BF16 GEMM, and dispatch receive. The GEMM is only a long covering window before receive; CUPTI directly measures the dispatch-send kernel reported in Figure 5a.

For each round we take the slowest-rank mean, then average across five separately executed rounds. Relative panels first compute the percentage change against default GDAKI within the same round and allocation. Every cell has five matched rounds; round identities and variation are retained in the underlying data.

b) Experiment 2: controlled GPU-wait sweep: This experiment asks how much time after GPU release reduces the work exposed at dispatch receive. We fix EP8 on two four-GPU nodes, top-$k = 8$, and dispatch cap 128, and sweep the same nine token counts. Requested waits are $\{0, 4, 8, 16, 32, 64, 96, 128, 256, 512, 1024\}$ μs. Each sequence launches separate dispatch-send, GPU-delay, and dispatch-receive kernels on the caller's compute stream. PyTorch's built-in delay uses one block and one thread; cycles are calibrated per GPU, and CUPTI records the actual delay in the same sequence. A no-delay sequence and a separate zero-cycle delay-kernel control record the delay kernel's duration and insertion effects. Figure 6 displays the eleven requested waits, including the no-delay case; the empty-kernel control remains in the evidence.

Each point captures 20 sequences in a CUDA graph after three ordinary warmup iterations. Three graph warmup replays precede three unprofiled and three CUDA-only CUPTI-profiled timed replays, retaining 60 paired send/receive samples per rank. A captured all-reduce on a dedicated NCCL control group aligns peer GPUs before external CUDA timing events bracket the 20 sequences; this alignment is outside the measured window. The default Gloo host-control group is not captured. Receive-count and BF16 round-trip checks, phase counts, stream placement, and launch geometry are validated for every point.

For each rank we average the same-invocation sum $T_{\mathrm{send}} + T_{\mathrm{receive}}$, take the largest of the eight rank means within a round, and report the median of five rounds. Because we time the two communication kernels directly, the delay itself is excluded. Actual delays, signed inter-kernel gaps, per-rank samples, and complete-sequence timing remain in the evidence. All 396 plotted backend/token/wait cells have five accepted rounds.

To check that CUPTI profiling does not distort these timings, the profiled and unprofiled CUDA-event means of every point must agree within the larger of 5 μs or 20% of the unprofiled mean; all reported points pass.

The GPU delay stands in for covering work without adding a GEMM's compute and memory traffic. The exposed-time floor includes send and receive execution and does not approach zero. We mark the shortest wait whose measured time is within 5% of the lowest value for that backend and token count.

B. vLLM TPOT Experiments

The native experiments measure GPU release and overlap directly. The vLLM experiments test whether real DBO supplies enough work to hide communication and reduce TPOT. We plug the same four NCCL-EP decode backends into vLLM and serve Qwen3-30B-A3B in BF16 on a 16-GPU GB300-CX8 NVL72 allocation: two four-GPU prefill nodes and two four-GPU decode nodes. We report median time per output token (TPOT). Every point averages three run medians; relative values use the common three-run default-GDAKI mean. The caller compute stream runs both low-latency send and receive rather than using a separate communication stream, matching the stream configuration used by DBO. Both serving roles set the dispatch/combine device-SM cap to 64 with four GIN contexts per rank, so serving does not run at the cap of 128 used in the native wait sweep and several native examples.

The workload uses synthetic random prompts with temperature 0 and `--ignore-eos`, so every request generates exactly its requested output length. Input length is nominal: the prompt generator can yield 127 tokens for a nominal length of 128; saved request records retain actual lengths. Each server session runs 64 warmup requests before measurement. At client concurrency C, at most C requests are in flight at once, and each point measures $4C$ requests (at least 64) with no additional request-rate limit.

We also include a DeepEP default-GDAKI serving baseline, using three runs at each matching concurrency and output length. The baseline is a DeepEP v1 low-latency (LL) integration fork based on DeepEP v1.2.1 and running over NCCL GIN, not the upstream NVSHMEM-based V1 release or DeepEP V2 [8]. We use V1 LL because its dispatch returns a receive hook: send and receive are separate calls on the caller compute stream, the same hook-mode release boundary used by our NCCL-EP configuration under DBO. Upstream DeepEP has since replaced V1 with a unified V2 interface and removed the V1 APIs. The model, precision, 2P2D topology, DBO threshold, decode graph mode, and intranode-only NVLink setting match the NCCL-EP study. DeepEP uses its default-GDAKI path on prefill and decode. The optimized configurations are evaluated within NCCL-EP; DeepEP contributes only its baseline. All serving percentages, including the DeepEP row, use the NCCL-EP default-GDAKI mean at the same point as their reference.

a) Experiment 1: concurrency sweep: This experiment fixes nominal input and output length at 128 tokens and sweeps client concurrency over $\{1, 32, 64, 128, 256, 512, 1000\}$. It combines four serving techniques:

1) Disaggregated serving. We use a 2P2D deployment so prefill and decode execute on separate nodes. This keeps prefill expert-parallel execution and its EP traffic off the decode nodes; NIXL KV-cache transfer from prefill to decode still uses the decode-side network rails and can overlap decode EP traffic. Every serving configuration uses default GDAKI on prefill; the evaluated backend runs on decode.

2) CUDA-graph replay. Decode uses the V1 model runner and `FULL_DECODE_ONLY` capture. Replay removes launch overhead and makes the communication interval a larger fraction of TPOT. With DBO enabled, the runner captures one CUDA graph per padded batch shape: a DBO-split graph if the shape reaches the DBO threshold, and an unsplit graph otherwise. A step at a split-captured shape that does not split at run time has no matching graph and falls back to eager execution; later split steps still replay the captured graph.

This single-graph behavior is intentional: eager execution makes cells that mix split and unsplit steps visible in TPOT. Capturing both split and unsplit graphs would hide that penalty without identifying how much each path contributes.

3) RDMA for inter-node traffic. NVLink remains enabled within each node, but cross-node NVLink is disabled. Local traffic therefore uses NVLink while every inter-node expert transfer uses the RDMA path under test.

4) Dual-batch overlap. DBO uses threshold 2 and two microbatches. Client concurrency is cluster-wide, whereas the split decision is made per step from the decode tokens scheduled on each of the eight data-parallel (DP) decode ranks, before MoE expert dispatch. A step splits only if every rank's scheduled count reaches the threshold and padding all ranks to the largest count would not leave the final microbatch of the least-loaded rank empty.

We set the threshold to 2 deliberately. It is the smallest value at which a step can split into two non-empty microbatches, so DBO engages across the whole concurrency sweep and its low-load boundary becomes visible. The sweep therefore also captures near-threshold steps, such as two scheduled tokens on a rank split into two one-token microbatches on the same caller compute stream. These splits leave almost no compute to cover communication and are not an economical operating point; a production deployment would use a higher threshold and would not run them.

Concurrency 1 is an important control: it never activates DBO, so every step replays its unsplit CUDA graph. At concurrency 32–64, variation in request placement and scheduled decode batch sizes can change the all-rank DBO vote or leave the final microbatch empty after common padding, selecting the non-DBO path on some steps, while another step at the same padded shape selects DBO. Those non-DBO steps run eagerly, as described above, and **we exclude these mixed cells from the transport comparison**. At concurrency 128–1000, each decode rank schedules well above the threshold, and run logs show both microbatches executing on every decode worker.

b) Experiment 2: output-length sweep: This experiment holds concurrency at 256 and nominal input length at 128, then sweeps output length over $\{128, 256, 512, 1024, 2048\}$ with the same four decode backends. Each backend has three runs, output lengths increase within each server session, and every point measures 1,024 requests. Figure 9 reports the mean of the three run-level median TPOTs, separately from the concurrency sweep.

c) Correctness checks: Because the optimizations change transfer and signal progression, timing is meaningful only if ordering and outputs remain correct. Every native cell checks

receive counts and a combine reference. For serving, we require zero failed requests and verify the outputs of every run against DeepEP. Nine DeepEP runs of the concurrency-256, 128/128 point, whose 1,024 prompts are identical in every run, form the reference. Because batched inference is not bitwise deterministic even at temperature 0, we compare the first two characters of each generated text, on the 949 prompts where all nine reference runs agree. A run passes if at most 3% of these prompts differ from the reference. To confirm that this check tracks task accuracy, we also evaluated separate runs with GSM8K (1,319 questions, 5-shot, greedy), including a configuration with a known output-corruption bug: the dma-buf address wrap that the KV-cache alignment guard prevents.

IV. Results

The results answer three questions. First, do GPI, GPI-MC, and GDAKI Free Flow release the GPU sooner than default GDAKI? Second, how much independent work is needed to hide the network progress that remains after release? Third, when DBO provides that work in vLLM, do the complete configurations reduce TPOT? We answer these questions in order with the dispatch-send, controlled-wait, and serving experiments.

A. NCCL-EP Dispatch-Send Latency

Figures 5a and 5b report the complete factorial: all 648 cells across 4, 8, and 16 GPUs have five matched rounds, 3,240 backend–round measurements in total, and every cell passes its receive-count and combine-reference checks.

The default-GDAKI panel shows the mean of the per-round slowest-rank mean kernel time. For another backend, we first compute its percentage change from default GDAKI within the same round and allocation, then average those paired percentages. Every comparison now has its same-round baseline, with the same five rounds used for absolute and relative values. Negative percentages mean earlier GPU release. Ranges and sample counts describe observed rounds rather than confidence intervals.

a) Backend regimes: Figures 5a and 5b show a regime transition rather than one universal winner. At 1–8 tokens/rank, GPI has the lowest displayed mean in 57 of the 72 GPU/top-k/SM-cap cells, with GPI-MC leading in the other 15 cells. As token count, top-k, and GPU count increase, GDAKI Free Flow takes over: it has the lowest displayed mean in 48 of the 72 cells from 32–256 tokens/rank, and in 25 of the 30 16-GPU cells from 16–256 tokens/rank; GPI leads in the other five.

The complete EP8, top-$k = 8$, cap-128 row illustrates the crossover. At two tokens/rank, default GDAKI, GPI, GPI-MC, and GDAKI Free Flow take 29.5, 14.5, 16.4, and 19.0 μs, respectively. At 128 tokens those times become 57.5, 111.0, 30.1, and 18.9 μs; at 256 tokens they become 107.1, 210.9, 101.3, and 25.7 μs.

GPI-MC therefore largely repairs GPI's high-token scaling, but pays a small low-token cost. Its compact fan-out publication delays the degradation of the ordinary GPI path. GDAKI Free Flow remains the more robust high-load path as system scale grows.

Across the completed factorial, GPI's mean paired dispatch-send time change is negative in 139 of 162 positions. GPI-MC improves 160 of 162 positions, including every top-$k = 8$ position; its only two exceptions are top-$k = 4$ cells on 8 GPUs at 256 tokens/rank. GDAKI Free Flow improves all 162 positions. These are descriptive counts over heterogeneous shapes, with five rounds per position, and do not by themselves establish a serving gain.

B. Exposed Communication Time under Controlled Waiting

Every cell in Figure 6 is absolute exposed communication time in μs.

Figure 6 answers two questions: how long network progress stays in flight after dispatch send releases the GPU, and how much exposed time remains once that progress is hidden. Along each row, exposed time falls as the GPU wait grows, because receive waits less for in-flight data, and then levels off once the wait covers the in-flight interval. The marked cell is therefore the covering window a backend needs: its in-flight time is approximately that wait. The level the row settles at, its floor, is the send and receive work that remains on the GPU however long the covering work is.

a) Offload lengthens the in-flight interval: Default GDAKI needs the shortest covering window: 16–32 μs up to 128 tokens/rank and 64 μs at 256. Its GPU builds WQEs and rings doorbells during send, so little work remains in flight after release. The offload paths move that work out of send and into the in-flight interval, so they need longer windows. GPI needs the longest: 64 μs already at 1–16 tokens/rank, 96 μs at 32, 256 μs at 64–128, and 512 μs at 256. Its window grows fastest with load, consistent with increasing per-destination submission overhead at high fan-out. GPI-MC, which requires fewer GPU descriptor submissions for fan-out, halves GPI's window at 128 and 256 tokens/rank (128 and 256 μs). GDAKI Free Flow keeps a short window across the sweep: 32–64 μs up to 128 tokens/rank and 96 μs at 256.

b) Once hidden, offload leaves less exposed time: With a long enough window, every offload path settles at a lower floor than default GDAKI. At 1–16 tokens/rank the floor is 22–24 μs for GPI and 24–27 μs for GPI-MC and Free Flow, against 34–41 μs for default GDAKI. At 256 tokens/rank default GDAKI still exposes 108.8 μs, against 35.5 μs for Free Flow and 75.6 μs for GPI-MC. GPI's floor rises to 180.5 μs at this load, indicating that the submission path remains GPU-visible during send, not only during subsequent network progress.

c) Implication: The two NCCL-EP experiments answer complementary questions. First, the in-flight interval only has to fit within the covering work: once it does, it is fully hidden, and a shorter interval brings no further gain. Default GDAKI's short window is therefore an advantage only when the covering work is shorter than the offload paths' windows, below 64 μs for GPI at small token counts and 32–96 μs for Free Flow. Second, once the in-flight interval is covered, exposed time is set by the send and receive work that stays on the GPU,

Top-$k = 8$

(a) Default GDAKI (μs)

GPUs / SM cap \ tokens/rank	1	2	4	8	16	32	64	128	256
4/32	35	37	35	38	40	45	60	90	145
4/64	34	35	34	36	39	44	54	81	133
4/128	34	34	35	37	39	44	52	68	117
8/32	31	32	32	34	37	40	55	86	161
8/64	29	31	30	33	34	38	46	71	152
8/128	29	30	30	32	34	38	45	57	107
16/32	27	28	30	32	35	39	63	152	327
16/64	27	27	30	32	34	41	50	101	246
16/128	27	28	28	31	32	40	51	60	148

(b) GPI (%)

GPUs / SM cap \ tokens/rank	1	2	4	8	16	32	64	128	256
4/32	-61%	-64%	-61%	-64%	-67%	-68%	-64%	-63%	-41%
4/64	-60%	-63%	-62%	-64%	-67%	-68%	-71%	-60%	-40%
4/128	-61%	-62%	-61%	-64%	-66%	-66%	-66%	-57%	-34%
8/32	-56%	-58%	-57%	-56%	-59%	-53%	-40%	+15%	+22%
8/64	-56%	-54%	-55%	-52%	-51%	-36%	-20%	+41%	+33%
8/128	-52%	-51%	-53%	-49%	-53%	-49%	-2%	+93%	+97%
16/32	-48%	-49%	-50%	-48%	-40%	-17%	+3%	-3%	-6%
16/64	-51%	-50%	-49%	-51%	-46%	-46%	+31%	+38%	+19%
16/128	-52%	-44%	-44%	-46%	-46%	-21%	+53%	+167%	+120%

(c) GPI-MC (%)

GPUs / SM cap \ tokens/rank	1	2	4	8	16	32	64	128	256
4/32	-64%	-65%	-63%	-64%	-66%	-69%	-68%	-63%	-60%
4/64	-62%	-61%	-63%	-64%	-66%	-65%	-70%	-76%	-61%
4/128	-57%	-62%	-63%	-65%	-65%	-67%	-74%	-80%	-57%
8/32	-51%	-49%	-52%	-56%	-54%	-56%	-51%	-41%	-30%
8/64	-49%	-51%	-49%	-50%	-50%	-49%	-54%	-36%	-15%
8/128	-42%	-45%	-43%	-46%	-50%	-52%	-49%	-48%	-5%
16/32	-35%	-38%	-34%	-35%	-35%	-44%	-46%	-63%	-60%
16/64	-40%	-37%	-44%	-35%	-46%	-48%	-43%	-36%	-42%
16/128	-35%	-40%	-35%	-38%	-35%	-40%	-47%	-18%	-16%

(d) GDAKI Free Flow (%)

GPUs / SM cap \ tokens/rank	1	2	4	8	16	32	64	128	256
4/32	-51%	-52%	-48%	-52%	-56%	-62%	-59%	-58%	-56%
4/64	-50%	-50%	-48%	-52%	-55%	-61%	-68%	-69%	-72%
4/128	-49%	-48%	-48%	-52%	-55%	-60%	-67%	-74%	-79%
8/32	-40%	-41%	-41%	-46%	-51%	-55%	-54%	-55%	-58%
8/64	-37%	-39%	-39%	-43%	-47%	-53%	-61%	-65%	-75%
8/128	-37%	-36%	-36%	-42%	-47%	-52%	-59%	-67%	-76%
16/32	-31%	-33%	-37%	-41%	-48%	-54%	-59%	-74%	-79%
16/64	-32%	-32%	-35%	-42%	-46%	-57%	-64%	-75%	-84%
16/128	-34%	-32%	-34%	-40%	-43%	-54%	-64%	-70%	-82%

tokens/rank

panel (a): 0 — 175 — 350 μs

(b)–(d); blue is better: -100% — 0% — $+100\%$

Fig. 5a. Five-round NCCL-EP dispatch-send sweep at top-$k = 8$. Columns vary tokens/rank; rows cross GPU count with dispatch CTA/SM cap. Panel (a) shows default-GDAKI dispatch-send time in μs. Panels (b)–(d) show mean paired percentage changes for GPI, GPI-MC, and GDAKI Free Flow, with a % sign in every relative cell. All 324 positions have five matched rounds. Relative colors saturate at $\pm 100\%$; printed values retain larger changes. NVLink is disabled. The matching top-$k = 4$ grid is in Appendix A, Figure 5b.

the floor in Figure 6. Its send component is the dispatch-send time of Section IV-A, which is why the paths that release the GPU earlier also reach lower floors. Section IV-C tests whether DBO in vLLM decode supplies enough covering work to hide the in-flight interval.

C. vLLM TPOT

Figure 7(a) reports the NCCL-EP default-GDAKI baseline in milliseconds and percentage changes for the DeepEP default-GDAKI baseline and three optimized NCCL-EP configurations, including the low-concurrency controls. Figure 7(b) reports absolute TPOT for these five series in the concurrency-128–1000 region where DBO operates reliably.

For each backend and concurrency we average three inde-

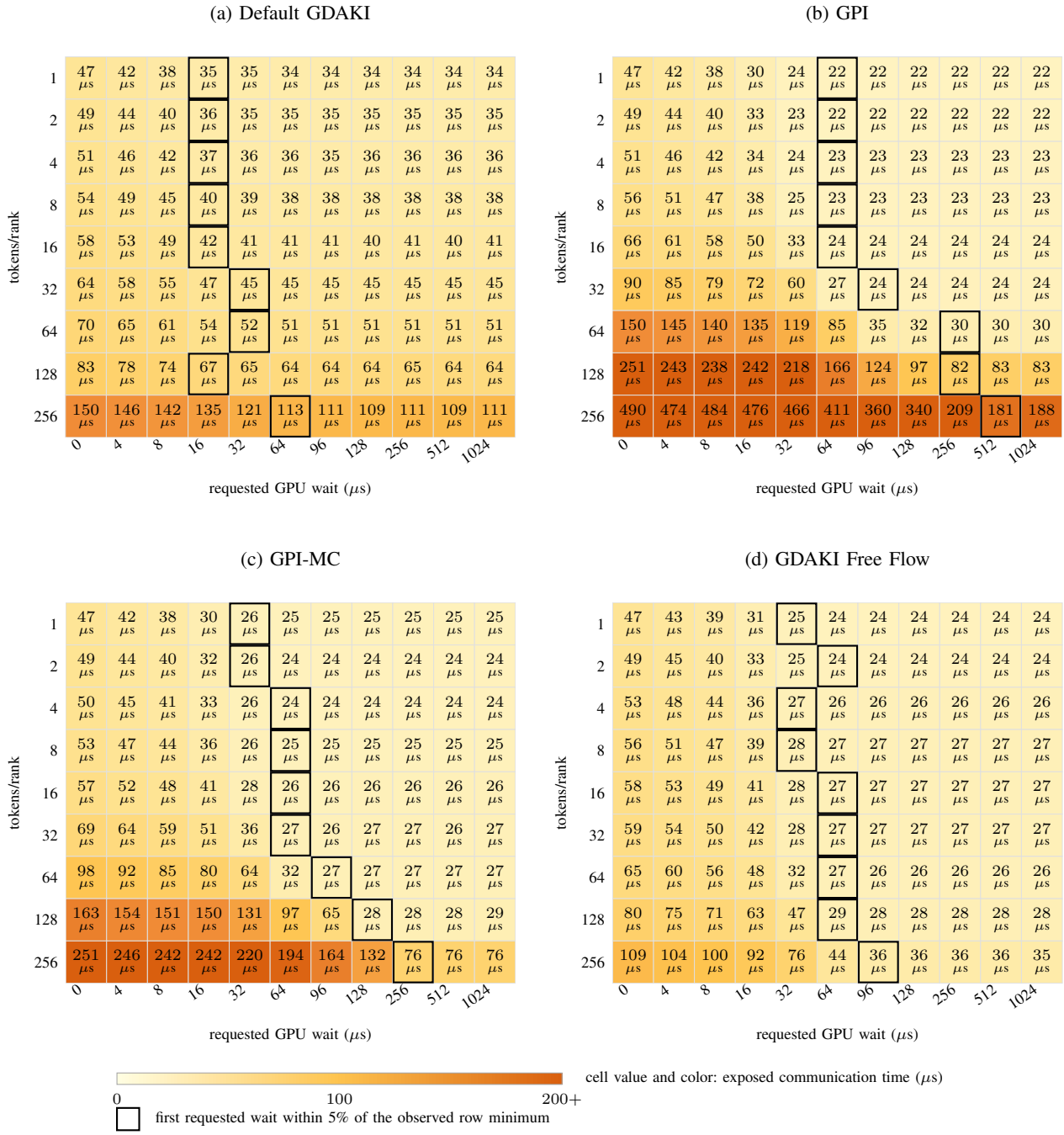


Fig. 6. Five-round NCCL-EP GPU-wait sweep at EP8, top-$k = 8$, dispatch cap 128, and NVLink off. The 2×2 panels show tokens/rank against requested GPU wait. Each cell is the median across five rounds of the slowest-rank mean of directly paired $T_{\text{send}} + T_{\text{receive}}$, in μs, excluding the delay kernel. Every cell includes its unit. Zero means no delay kernel; the separate empty-kernel control is retained in the data. Requested waits are calibrated per GPU and actual in-sequence delays are recorded. These are communication-kernel durations, not differences of standalone timings. A solid outline marks the first requested wait within 5% of that row's lowest observed median; it is an empirical near-floor marker.

pendent run-level median TPOT values, then normalize to the same three-run NCCL-EP default-GDAKI mean. Percentages are ratios of these means.

a) Correctness: Every NCCL-EP serving run completes with zero failed requests and passes the output check, with 0.11–0.84% of the 949 reference prompts differing from DeepEP (Figure 8(b)). The check tracks task accuracy (Figure 8(a)): on the GSM8K validation runs, correct configurations scored 0.884–0.890, matching DeepEP (0.888) and a single GPU (0.89), with at most 0.9% differing, whereas configurations with the dma-buf address-wrap bug scored 0.526–0.644 with 12.4–34.9% differing.

b) Reliable-DBO region: At concurrency 128–1000, all three optimized paths lower median TPOT by 15–25% relative

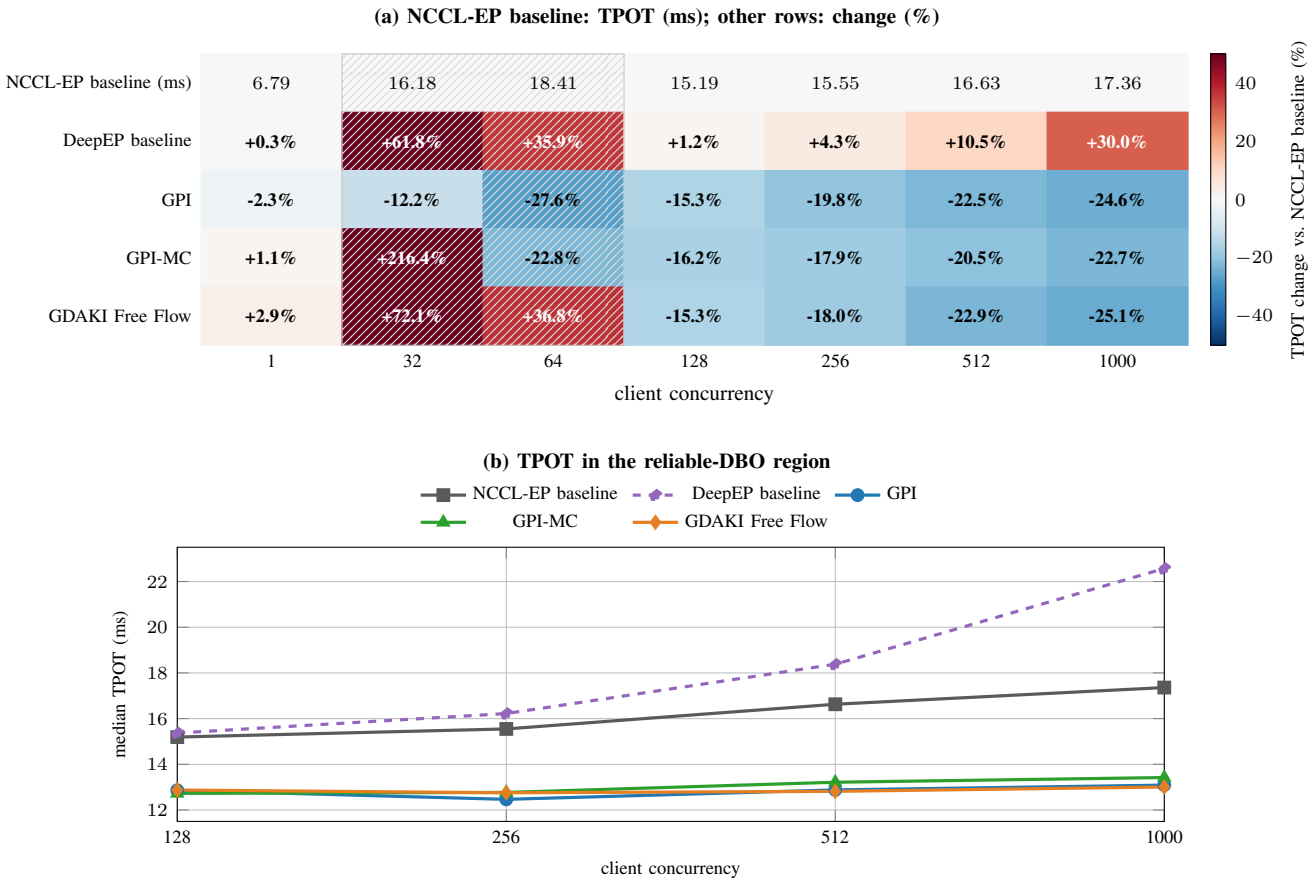


Fig. 7. Measured vLLM TPOT for nominal 128 input and 128 output tokens. (a) The NCCL-EP baseline shows actual median TPOT in milliseconds; the DeepEP baseline and optimized NCCL-EP rows show percentage changes from that NCCL-EP baseline across the full concurrency sweep. Concurrency 1 is a stable non-DBO control. **The hatched concurrency-32/64 region is not transport evidence: variation in scheduled decode batch sizes across ranks can fail the all-rank DBO vote on some steps, and those non-DBO steps have no captured graph and run eagerly. The single graph per shape is intentional, so that this penalty exposes cells that mix DBO and non-DBO steps (Section III-B). Splitting roughly two tokens into one-token microbatches is also not an economical DBO regime.** (b) Absolute median TPOT for all five series in the reliable-DBO region. Both baseline series use default GDAKI. Every plotted value is the mean of three run medians, and every relative value uses the same three-run NCCL-EP reference. All configurations use default GDAKI prefill.

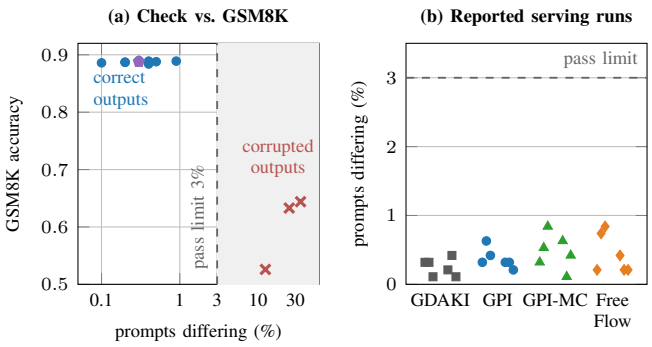


Fig. 8. Output correctness. (a) GSM8K accuracy against the share of the 949 reference prompts whose output differs from DeepEP, for validation runs of correct configurations (blue), DeepEP itself (purple), and configurations with the dma-buf address-wrap bug (red). (b) The same check for every NCCL-EP serving run reported in this paper, three runs per backend for each of the two sweeps. The dashed line marks the 3% pass limit.

to default GDAKI: GPI by 15.3–24.6%, GPI-MC by 16.2–22.7%, and GDAKI Free Flow by 15.3–25.1%. For every path the reduction grows with concurrency. At every one of these points, the slowest run of each optimized path is faster than the fastest default-GDAKI run. The relative ranking of the three paths varies with concurrency.

c) DeepEP baseline comparison: At concurrency 128–1000, the DeepEP default-GDAKI baseline is 1.2–30.0% slower than the NCCL-EP baseline, and the gap widens with concurrency. Both baselines are shown explicitly, so the comparison separates the library baseline difference from the gains of the optimized NCCL-EP paths.

d) Low-concurrency controls: At concurrency 1, vLLM never splits the decode batch. CUDA-graph replay is stable because the step is always unsplit, but there is no second microbatch behind which to hide network progress. Accordingly, the three optimized configurations stay within −2.3% to +2.9% of the GDAKI baseline; this cell does not test the DBO-overlap claim.

At concurrency 32–64, the decode tokens scheduled on each DP rank lie near DBO's two-token threshold. Variation in request placement and scheduled batch sizes can make one step fail the all-rank vote, because some rank falls below the threshold or because padding to the largest rank would leave the final microbatch empty, selecting the non-DBO path, while another step passes it and selects DBO. Because we capture only one CUDA graph per padded shape, in the split state, each non-DBO step falls back to eager decode. This large, backend-independent penalty is the intended signal of the single-graph design (Section III-B): it marks these cells as mixtures of DBO and non-DBO steps, which are unsuitable for comparing network backends. Capturing a second, unsplit graph would hide the penalty but leave the mixture, so the measured TPOT could no longer be attributed to DBO.

The regime is also not a useful target for DBO. With roughly two tokens on a rank, DBO creates two one-token microbatches on the same compute stream, leaving almost no compute behind which to hide communication while still paying the split, scheduling, and synchronization overhead. We do not treat such splits as an economical DBO configuration. Concurrency 32–64 is therefore excluded as transport evidence for two reasons: its steps mix split and unsplit execution, and its near-threshold splits are too small to benefit from overlap.

e) Boundary: The result does not show that enabling DBO is always an absolute win. It requires enough per-rank work to amortize the split and expose a useful covering window. The supported result is narrower: at the concurrency points where DBO splits reliably and economically, the optimized configurations have lower median TPOT, consistent with earlier release improving overlap. The serving experiment compares whole configurations; it does not isolate dispatch-send duration, receive/combine cost, or final remote-arrival time. Unchanged or slower network completion remains a conditional scheduling argument, rather than a measured serving result.

f) Output-length sensitivity: Figure 9 holds client concurrency at 256 and nominal input length at 128 tokens, then varies output length from 128 to 2048 tokens. The NCCL-EP baseline ranges from 15.64 to 16.34 ms, while GPI, GPI-MC, and GDAKI Free Flow remain within 12.49–13.06, 12.74–13.37, and 12.71–13.24 ms, respectively. All three configurations retain their median-TPOT advantage across the measured output lengths. The DeepEP baseline ranges from 16.37 to 16.87 ms, or 2.4–6.8% above the NCCL-EP baseline at the corresponding output lengths.

The three features therefore sustain their benefit for workloads with reliable DBO and sufficient progress resources.

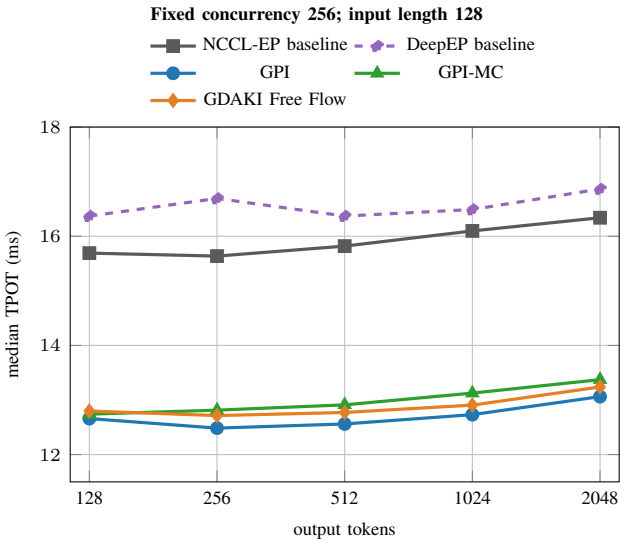


Fig. 9. Measured vLLM output-length sweep at fixed concurrency 256 and nominal input length 128. Each value is the mean of three run-level median TPOTs, with 1,024 measured requests per point. Prefill is default GDAKI in every configuration. The additional DeepEP default-GDAKI baseline uses three runs per point.

V. Discussion

A. Optimize Exposed Communication Time

The experiments support a scheduling principle:

> *Minimize exposed communication time by releasing the GPU early, then assign remaining progress to an engine that can run beneath useful work.*

This objective is more precise than minimizing one isolated transport call or the sequential dispatch/combine total. The two base approaches place network-side progress across different submission boundaries: GPI has the GPU submit GFDs to NIC-accessible memory for transport, while GDAKI Free Flow leaves WQE construction on the GPU but moves ready-prefix serialization and doorbells to a host progress thread. GPI-MC is the experimental MoE refinement of the first approach; it amortizes fan-out publication so GPI retains its ordinary behavior at higher token counts. The controlled wait sweep shows the progress opportunity after release, while the serving experiment measures the TPOT of the complete configurations with a common default-GDAKI prefill backend.

The principle allows an initially counterintuitive trade. Suppose backend b releases the GPU Δ earlier but has the same or slightly longer time until the final remote arrival. By (1), work from the other microbatch starts up to Δ earlier, and as long as the final remote arrival falls within that covering work, receive does not wait on the network and the layer finishes earlier. In practice the covering work is long enough: the in-flight windows in Section IV-B are 16–512 μs, and in vLLM decode all three optimized paths lower TPOT at every reliable-DBO point. Under DBO, dispatch-send time, not network completion time, is therefore the quantity to minimize.

a) The optimization objective determines the preferred path.: These results are not a universal ranking of GIN backends. Under DBO, the objective is to minimize exposed communication time by releasing the GPU early enough for another microbatch to run, while network-side progress continues beneath that work. Without DBO or another source of covering work, the relevant objective becomes the end-to-end latency from dispatch send through in-flight progress

to dispatch receive. We do not evaluate that regime, and the preferred backend there may differ. The evaluated prototype therefore uses the three configurations as complementary options: GPI submits per-destination GFDs for NIC transport, GPI-MC describes token fan-out in one GFD, and GDAKI Free Flow provides host-assisted progress. Selection depends on workload, available NIC or host progress resources, and which communication intervals the application can hide.

B. Limitations

GPU wait as covering work. In a real workload, the other microbatch launches expert GEMMs between dispatch send and receive. In time, the requested wait stands in for that covering work: a GEMM that lasts as long as the wait gives the network the same progress window, plus some launch and scheduling overhead. What the wait does not reproduce is resource contention. A GEMM occupies SMs and competes with the communication kernels for registers, shared memory, and memory bandwidth, which the one-block, one-thread delay does not. Under that contention, send, receive, and GPU-side progress may run slower than in the wait sweep.

Serving controls. Each backend and point has three runs. The serving experiment swaps complete backends, so a TPOT change reflects the whole backend, including its receive and combine paths, not dispatch send alone; the native experiments identify earlier GPU release with hidden in-flight progress as the mechanism behind it. DeepEP appears only as a default-GDAKI serving baseline.

Topology and scale. The native experiments cover 4, 8, and 16 GPUs over one, two, and four nodes, and the serving experiments use two four-GPU decode nodes serving Qwen3-30B-A3B with 128 input tokens, up to 2,048 output tokens, and client concurrency up to 1,000. Production deployments differ along every one of these axes: longer inputs and outputs, decode groups of 20 or more nodes, higher concurrency, and larger models. Each changes the share of TPOT that dispatch communication occupies, and therefore how much earlier GPU release matters. Larger decode groups spread tokens across more remote peers and may make network progress a larger share of each step; larger models and longer sequences add expert and attention compute that may make it a smaller share. We do not measure which effect dominates, so the gains reported here may grow or shrink in production. Intranode-only NVLink is a deliberate mixed path and does not model a fully connected NVL72 deployment.

GPI-MC is still experimental. GPI-MC is an early prototype built on GPI, and its fan-out submission path and high-load behavior have not yet been tuned. The results here are therefore a starting point rather than its best achievable performance.

CPU progress capacity. GDAKI Free Flow depends on a host thread that continuously polls for ready WQEs. Its gains therefore assume sufficient host CPU capacity for timely progress. In CPU-intensive serving pipelines—including agentic workloads with substantial host-side orchestration, tokenization, retrieval, or tool-processing overhead—contention for that CPU capacity may delay network progress, reduce the observed benefit, or limit scaling. We did not evaluate CPU-saturated conditions or affinity sensitivity, so the results should not be extrapolated to those regimes. In contrast, GPI's steady-state send path does not require a CPU progress thread: after the GPU publishes a GFD, the NIC progresses the operation.

PCIe polling contention. On systems where the host reads GPU-resident readiness state across PCIe, Free Flow polling adds PCIe transactions. When PCIe is already busy, those transactions add pressure and may take longer to complete, further delaying network progress. We did not evaluate PCIe contention or sensitivity to ready-state memory placement, so the results should not be extrapolated to systems with a saturated host–device path. GPI does not generate this continuous host-polling traffic. For each submitted operation, it publishes a compact GFD to NIC-accessible memory and then returns. This one-time descriptor write does not create sustained PCIe pressure.

The other three communication phases. This paper optimizes only dispatch send. Exposed communication time also includes dispatch receive, combine send, and combine receive, and each can be reduced in its own way: combine send can apply the same early-release principle, while the two receive phases determine how much GPU work remains once data arrives. Optimizing all four toward minimal exposed communication time is future work.

Alternative schedules. All serving results use single-stream DBO: both microbatches and NCCL-EP send and receive share the caller's compute stream. We did not compare against two other schedules: running communication, or each microbatch, on its own CUDA stream, and serving without DBO. Either could be faster than single-stream DBO for some workloads. If so, the earlier release we optimize for would matter less, so both comparisons need future experiments.

VI. Related Work

GPU-initiated networking. GPUrdma and GPUDirect Async remove the CPU from the communication critical path [9], [10]. GPU-initiated OpenSHMEM and NVSHMEM provide in-kernel one-sided communication and collectives [11], [12], DOCA GPUNetIO exposes NIC queues and doorbells to GPU kernels [7], and NCCL GIN integrates device-initiated operations with NCCL communicators [5], [13]. Groves et al. compare device-initiated and host-initiated NVSHMEM communication [14]. Baydamirli et al. dissect GPU-initiated communication at the GPU–NIC boundary across NVSHMEM, NCCL GIN, DeepEP, and other libraries, finding that libraries add up to 4.6 μs of issue time through queue management, memory ordering, and completion scope, and that a tuned CPU proxy matches the GPU path at idle at the cost of a dedicated core [15]. These studies characterize the submission path itself; we ask how its placement changes exposed communication time under DBO.

Offloading communication progress. BluesMPI offloads the progress of non-blocking MPI Alltoall to BlueField DPU cores to free the host for computation [16]. UCCL-EP keeps

GPU-initiated token routing but delegates RDMA issue to CPU proxy threads for portability across GPUs and NICs [17]. GPI and GDAKI Free Flow follow the same principle of moving progress off the critical resource, applied to the GPU SM: GPI has the GPU submit GFDs to NIC-accessible memory for transport, and Free Flow moves send progression to a host thread.

MoE communication and overlap. Sparse MoE models route each token to experts sharded across devices, creating data-dependent all-to-all traffic [1]–[3]. FasterMoE replicates popular experts and overlaps fine-grained all-to-all with expert computation [18], and Tutel adapts parallelism and pipelining at runtime and optimizes all-to-all [19]. A line of work hides MoE communication behind computation: Lancet overlaps all-to-all with the whole training graph [20]; Comet and MegaScale-MoE fuse tile-level communication into MoE compute kernels [21], [22], and TileLink generalizes such fusion into a compiler for overlapped compute–communication kernels [23]; and FlashMoE runs dispatch, expert computation, and combine in a single persistent kernel [24]. DeepSeek-V3 introduces specialized cross-node dispatch and combine kernels that overlap with computation [25], and DeepEP releases such kernels as a library whose low-latency mode adds receive hooks for overlap without occupying SMs [8]. NCCL-EP provides the expert-parallel interface over NCCL used here [6]. These systems change *what* overlaps with communication; we keep the overlap schedule fixed and reduce the GPU time that dispatch send itself consumes, which is complementary.

Microbatch overlap. vLLM manages the KV cache with PagedAttention and uses iteration-level (continuous) batching [26], and disaggregated systems separate prefill and decode [27], [28]. In training, DeepSeek-V3's DualPipe pairs a forward and a backward chunk, each split into attention, all-to-all dispatch, MLP, and all-to-all combine, so that one chunk's all-to-all runs beneath the other's computation [25]. In serving, DeepSeek's dual-batch overlap [4], MegaScale-Infer's ping-pong pipeline between attention and expert nodes [29], and NanoFlow's nano-batching [30] similarly split a batch so that one part's communication or memory-bound work runs beneath another part's computation. Our results show that, within such a schedule, the time communication holds the GPU before releasing it determines how much of the overlap is realized.

VII. Conclusion

MoE communication should be optimized at the boundary the scheduler sees: how long dispatch send holds the GPU. GPI has the GPU submit GFDs to NIC-accessible memory for transport, GPI-MC describes token fan-out in one GFD, and GDAKI Free Flow moves send progression to a host thread. All three release the GPU earlier than default GDAKI across most of the measured sweep. Offload lengthens the in-flight interval, but under DBO the other microbatch's work covers it, and what remains exposed is the shorter GPU-side work. In vLLM decode, this lowers median TPOT by 15–25% relative to NCCL-EP default GDAKI and by 16–42% relative to DeepEP v1 wherever DBO splits reliably. Together, the evaluated configurations demonstrate complementary ways to minimize exposed communication time under DBO.

References

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. V. Le, G. E. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://arxiv.org/abs/1701.06538

[2] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "GShard: Scaling giant models with conditional computation and automatic sharding," in *International Conference on Learning Representations (ICLR)*, 2021.

[3] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[4] DeepSeek-AI, "Day 6: One more thing, DeepSeek-V3/R1 inference system overview," Open Infra Index, 2025, accessed: 2026-10-29; text and figures released under CC0 1.0. [Online]. Available: https://github.com/deepseek-ai/open-infra-index/blob/main/202502OpenSourceWeek/day_6_one_more_thing_deepseekV3R1_inference_system_overview.md

[5] K. Hamidouche, J. Bachan, P. Markthub, P.-J. Gootzen, E. Agostini, S. Jeaugey, A. Shafi, G. Theodorakis, and M. G. Venkata, "GPU-Initiated Networking for NCCL," *arXiv preprint arXiv:2511.15076*, 2025. [Online]. Available: https://arxiv.org/abs/2511.15076

[6] A. Goldman, N. Boker, M. Sheraizin, N. Admoni, A. Polyakov, S. Bhattacharya, F. Yu, K. Sun, G. Theodorakis, H.-C. Yin, P.-J. Gootzen, A. Shafi, A. Ravid, S. Di Girolamo, J. Dinan, X. Li, M. G. Venkata, and G. Bloch, "NCCL EP: Towards a unified expert parallel communication API for NCCL," *arXiv preprint arXiv:2603.13606*, 2026. [Online]. Available: https://arxiv.org/abs/2603.13606

[7] NVIDIA Corporation, "DOCA GPUNetIO," NVIDIA DOCA SDK Documentation, v3.5.0, 2026, accessed: 2026-10-21. [Online]. Available: https://networking-docs.nvidia.com/doca/sdk/doca-gpunetio

[8] C. Zhao, S. Zhou, L. Zhang, C. Deng, Z. Xu, Y. Liu, K. Yu, J. Li, and L. Zhao, "DeepEP: An efficient expert-parallel communication library (v1.2.1)," GitHub repository, tag v1.2.1, 2025, accessed: 2026-10-21. [Online]. Available: https://github.com/deepseek-ai/DeepEP/tree/v1.2.1

[9] F. Daoud, A. Watad, and M. Silberstein, "GPUrdma: GPU-side library for high performance networking from GPU kernels," in *Proceedings of the 6th International Workshop on Runtime and Operating Systems for Supercomputers*, 2016.

[10] E. Agostini, D. Rossetti, and S. Potluri, "GPUDirect Async: Exploring GPU synchronous communication techniques for InfiniBand clusters," *Journal of Parallel and Distributed Computing*, vol. 114, pp. 28–45, 2018.

[11] K. Hamidouche and M. LeBeane, "GPU initiated OpenSHMEM: Correct and efficient intra-kernel networking for dGPUs," in *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2020, pp. 336–347.

[12] NVIDIA Corporation, "NVIDIA OpenSHMEM Library (NVSHMEM) documentation," NVIDIA Documentation, version 3.8.0, 2026, accessed: 2026-11-02. [Online]. Available: https://docs.nvidia.com/nvshmem/api/latest/index.html

[13] ——, "Device-initiated communication," NCCL User Guide, 2026, accessed: 2026-10-21. [Online]. Available: https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/deviceapi.html

[14] T. Groves, B. Brock, Y. Chen, K. Z. Ibrahim, L. Oliker, N. J. Wright, S. Williams, and K. Yelick, "Performance trade-offs in GPU communication: A study of host and device-initiated approaches," in *2020 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2020, pp. 126–137.

[15] J. Baydamirli, I. Ismayilov, K. Oktay, and D. Unat, "GPU-initiated communication: Dissecting down to the bone," *arXiv preprint arXiv:2610.01380*, 2026. [Online]. Available: https://arxiv.org/abs/2610.01380

[16] M. Bayatpour, N. Sarkauskas, H. Subramoni, J. M. Hashmi, and D. K. Panda, "BluesMPI: Efficient MPI non-blocking alltoall offloading designs on modern BlueField smart NICs," in *High Performance Computing: 36th International Conference, ISC High Performance 2021*, ser. Lecture Notes in Computer Science, vol. 12728, 2021, pp. 18–37.

[17] Z. Mao, Y. Zhang, C. Cui, Z. Huang, K. You, Z. Chen, Z. Xu, Z. Gu, S. Shenker, C. Raiciu, Y. Zhou, and I. Stoica, "UEP: Portable expert-parallel communication," in *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, 2026, also arXiv:2512.19849 as "UCCL-EP".

[18] J. He, J. Zhai, T. Antunes, H. Wang, F. Luo, S. Shi, and Q. Li, "FasterMoE: Modeling and optimizing training of large-scale dynamic pre-trained models," in *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2022, pp. 120–134.

[19] C. Hwang, W. Cui, Y. Xiong, Z. Yang, Z. Liu, H. Hu, Z. Wang, R. Salas, J. Jose, P. Ram, H. Chau, P. Cheng, F. Yang, M. Yang, and Y. Xiong, "Tutel: Adaptive mixture-of-experts at scale," in *Proceedings of Machine Learning and Systems*, vol. 5, 2023, pp. 269–287.

[20] C. Jiang, Y. Tian, Z. Jia, S. Zheng, C. Wu, and Y. Wang, "Lancet: Accelerating mixture-of-experts training via whole graph computation-communication overlapping," in *Proceedings of Machine Learning and Systems*, vol. 6, 2024, pp. 74–86.

[21] S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S. Zheng, L.-W. Chang, Q. Chen, and X. Liu, "COMET: Fine-grained computation-communication overlapping for mixture-of-experts," in *Proceedings of Machine Learning and Systems*, vol. 7, 2025.

[22] C. Jin, Z. Jiang, Z. Bai, Z. Zhong, J. Liu, X. Li, N. Zheng, X. Wang, C. Xie, Q. Huang *et al.*, "MegaScale-MoE: Large-scale communication-efficient training of mixture-of-experts models in production," in *Proceedings of the 21st European Conference on Computer Systems (EuroSys)*, 2026, pp. 366–382.

[23] S. Zheng, J. Fang, X. Zheng, Q. Hou, W. Bao, N. Zheng, Z. Jiang, D. Wang, J. Ye, H. Lin, L.-W. Chang, and X. Liu, "TileLink: Generating efficient compute-communication overlapping kernels using tile-centric primitives," in *Proceedings of Machine Learning and Systems*, vol. 7, 2025.

[24] O. J. Aimuyo, B. Oh, and R. Singh, "FlashMoE: Fast distributed MoE in a single kernel," in *Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 100 676–100 699.

[25] DeepSeek-AI, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024. [Online]. Available: https://arxiv.org/abs/2412.19437

[26] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 2023, pp. 611–626.

[27] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 193–210.

[28] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot," in *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, 2025, pp. 155–170.

[29] R. Zhu, Z. Jiang, C. Jin, P. Wu, C. A. Stuardo, D. Wang, X. Zhang, H. Zhou, H. Wei, Y. Cheng *et al.*, "MegaScale-Infer: Efficient mixture-of-experts model serving with disaggregated expert parallelism," in *Proceedings of the ACM SIGCOMM 2025 Conference*, 2025, pp. 592–608.

[30] K. Zhu, Y. Gao, Y. Zhao, L. Zhao, G. Zuo, Y. Gu, D. Xie *et al.*, "NanoFlow: Towards optimal large language model serving throughput," in *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, 2025, pp. 749–765.

APPENDIX A
TOP-$k = 4$ DISPATCH-SEND RESULTS

Figure 5b completes the dispatch factorial with its top-$k = 4$ slice. It uses the same five-round estimator, axes, colors, and cell units as the top-$k = 8$ result in Figure 5a.

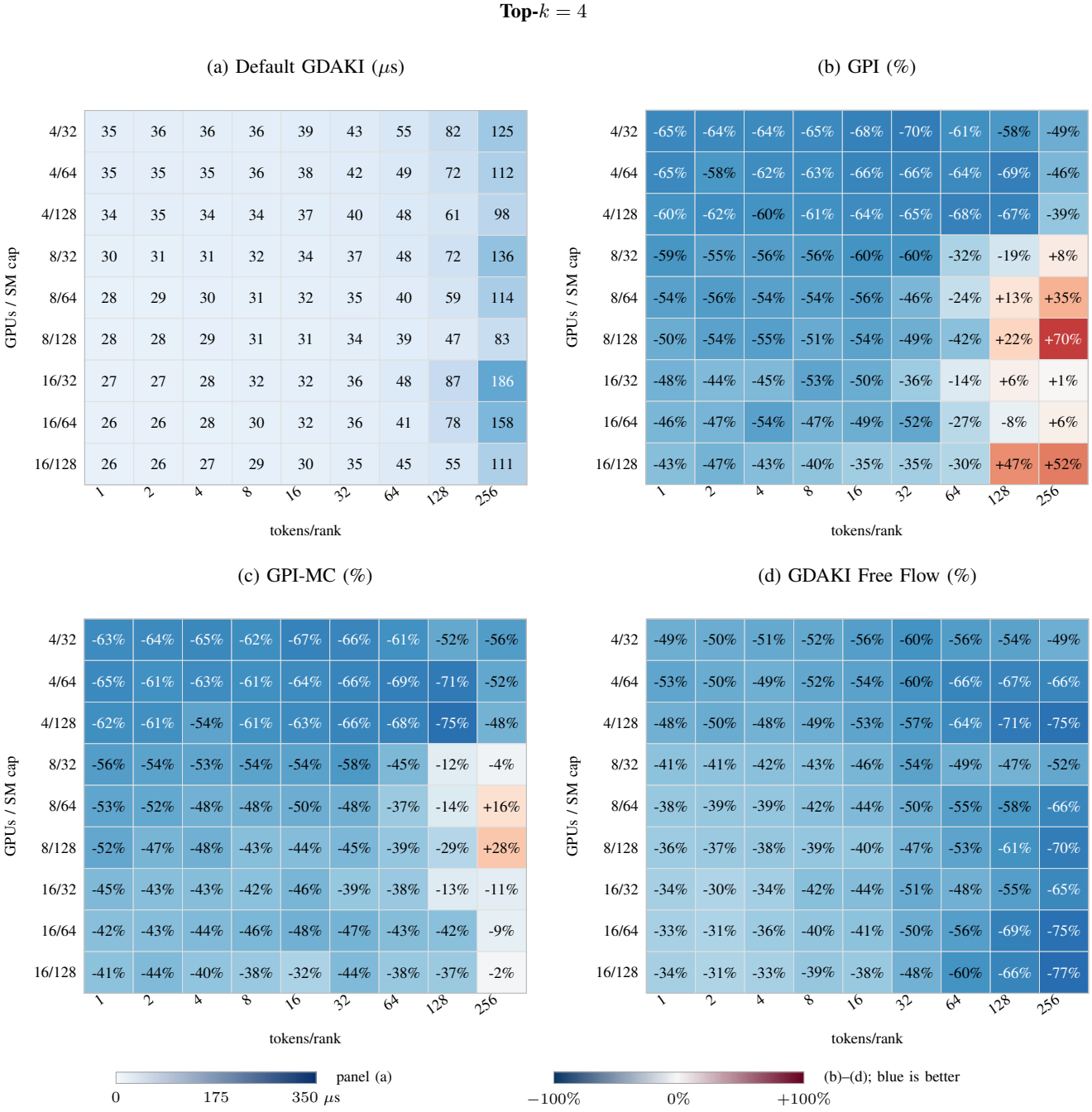


Fig. 5b. Five-round NCCL-EP dispatch-send sweep at top-$k = 4$. The 2×2 panel layout matches Figure 5a: (a) default-GDAKI time in μs; (b)–(d) paired changes for GPI, GPI-MC, and GDAKI Free Flow, with % signs in every relative cell. All 324 positions have five matched rounds. Colors saturate at $\pm 100\%$ for relative panels; printed values retain larger changes. NVLink is disabled.